

```
nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Fri_Feb_21_20:23:50_PST_2025
Cuda compilation tools, release 12.8, V12.8.93
Build cuda_12.8.r12.8/compiler.35583870_0
```

```
!nvidia-smi
```

```
Tue May 12 06:59:40 2026
```

```
-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+-----+
|  0   Tesla T4               Off   | 00000000:00:04:0 Off |                    0 |
| N/A   44C    P8              10W / 70W |  0MiB / 15360MiB |      0%      Default |
+-----+-----+-----+-----+-----+-----+
|                                           | N/A            |
```

```
-----+
| Processes:
| GPU   GI    CI          PID    Type   Process name                      GPU Memory
|   ID   ID                                     Usage
+-----+-----+-----+-----+-----+-----+
| No running processes found
+-----+-----+-----+-----+-----+-----+
|
```

```
%%writefile vecadd.cu
#include<bits/stdc++.h>
#include<chrono>
#include<cuda_runtime.h>

using namespace std;
using namespace std::chrono;

__global__ void add_gpu(int *a,int *b,int *c,int size){
    auto tid = blockIdx.x * blockDim.x + threadIdx.x;
    if(tid<size){
        c[tid] = a[tid]+b[tid];
    }
}

void add_cpu(int *a,int *b,int *c,int size){
    for(int i=0;i<size;i++){
        c[i]=a[i]+b[i];
    }
}

void initialize(int *a,int size){
    for(int i=0;i<size;i++){
        a[i]=rand()%100;
    }
}

void print_sample(int*a,int size){
    for(int i=0;i<min(size,10);i++){
        cout<<a[i]<<" ";
    }
    cout<<endl;
}

int main(){
    int size = 100000;
    int *a = new int[size];
    int *b = new int[size];
    int *c = new int[size];
    int *x,*y,*z;
    initialize(a,size);
    initialize(b,size);
    print_sample(a,size);
    print_sample(b,size);
```

```

        cudaMalloc(&x,size*sizeof(int));
        cudaMalloc(&y,size*sizeof(int));
        cudaMalloc(&z,size*sizeof(int));
        cudaMemcpy(x,a,size*sizeof(int),cudaMemcpyHostToDevice);
        cudaMemcpy(y,b,size*sizeof(int),cudaMemcpyHostToDevice);
        auto t1 =high_resolution_clock::now();
        add_cpu(a,b,c,size);
        auto t2 =high_resolution_clock::now();
        print_sample(c,size);
        int threads = 256;
        int blocks = (size+threads -1)/threads;
        auto t3 =high_resolution_clock::now();
        add_gpu<<blocks,threads>>>(x,y,z,size);
        cudaDeviceSynchronize();
        auto t4 =high_resolution_clock::now();
        cudaMemcpy(c,z,size*sizeof(int),cudaMemcpyDeviceToHost);
        print_sample(c,size);

        cout<<"Time req for cpu : "<<duration_cast<microseconds>(t2-t1).count()<<endl;
        cout<<"Time req for gpu : "<<duration_cast<microseconds>(t4-t3).count()<<endl;
        delete []a;
        delete []b;
        delete []c;
        cudaFree(x);
        cudaFree(y);
        cudaFree(z);

        return 0;
    }

```

Overwriting vecadd.cu

!nvcc vecadd.cu -o vecadd

nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future

!./vecadd

```

83 86 77 15 93 35 86 92 49 21
42 18 16 16 63 6 28 66 39 41
125 104 93 31 156 41 114 158 88 62
125 104 93 31 156 41 114 158 88 62
Time req for cpu : 505
Time req for gpu : 191

```

```

%%writefile mat.cu
#include<bits/stdc++.h>
#include<chrono>
#include<cuda_runtime.h>

using namespace std;
using namespace std::chrono;

__global__ void gpu_mul(int *a,int *b,int *c,int M,int N,int K){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if(row<M&&col<K){
        int sum=0;
        for(int i=0;i<N;i++){
            sum += a[row*N+i]*b[i*K+col];
        }
        c[row*K+col]=sum;
    }
}

void cpu_mul(int *a,int *b,int *c,int M,int N,int K){
    for(int i=0;i<M;i++){
        for(int j=0;j<K;j++){
            int sum=0;
            for(int k=0;k<N;k++){
                sum+= a[i*N+k]*b[k*K+j];
            }
            c[i*K+j]=sum;
        }
    }
}

```

```

void initialize_arr(int *a,int m,int n){
    for(int i=0;i<m*n;i++){
        a[i]=rand()%100;
    }
}

int main(){
    int M=512,N=512,K=512;
    int *a= new int[M*N];
    int *b= new int[N*K];
    int *c= new int[M*K];
    int thrd = 16;
    initialize_arr(a,M,N);
    initialize_arr(b,N,K);
    dim3 threads(thrd,thrd);
    dim3 blocks(
        (thrd+K-1)/thrd,
        (thrd+M-1)/thrd
    );

    int *x,*y,*z;
    cudaMalloc(&x,M*N*sizeof(int));
    cudaMalloc(&y,N*K*sizeof(int));
    cudaMalloc(&z,M*K*sizeof(int));

    cudaMemcpy(x,a,M*N*sizeof(int),cudaMemcpyHostToDevice);
    cudaMemcpy(y,b,N*K*sizeof(int),cudaMemcpyHostToDevice);

    auto t1 = high_resolution_clock::now();
    cpu_mul(a,b,c,M,N,K);
    auto t2 = high_resolution_clock::now();
    cout<<"Time Req for CPU : "<<duration_cast<microseconds>(t2-t1).count()<<endl;

    auto t3 = high_resolution_clock::now();
    gpu_mul<<<blocks,threads>>>(x,y,z,M,N,K);
    cudaDeviceSynchronize();
    cudaMemcpy(c,z,M*K*sizeof(int),cudaMemcpyDeviceToHost);
    auto t4 = high_resolution_clock::now();
    cout<<"Time Req for GPU : "<<duration_cast<microseconds>(t4-t3).count()<<endl;

    delete []a;
    delete []b;
    delete []c;

    cudaFree(x);
    cudaFree(y);
    cudaFree(z);

    return 0;
}

```

Overwriting mat.cu

```
!nvcc mat.cu -o mat
```

nvcc warning : Support for offline compilation for architectures prior to 'compute/sm/lto_75' will be removed in a future

```
!./mat
```

```
Time Req for CPU : 1355042
Time Req for GPU : 1451
```

Start coding or [generate](#) with AI.

